C Real Numbers

CSC 230 : C and Software Tools

NC State Department of Computer Science

Topics for Today

- C Math Library
- Floating Point Representation

Math

- Common math support is deckared in math.h

```
#include <math.h>
```

- You also need to link with the math library:

```
gcc -Wall -std=c99 program.c -o program -lm
```

- You get some important constants

Macro	Value
M_E	Base of the natural logarithm
M_PI	You know, π
M_SQRT2	Square root of 2

Trigonometry

- You get common mathematical functions

```
double sin(double x);
```

Double goes in

Double comes out.

- And lots of friends:

Function	Description
sin(x)	Trigonometric sine, cosine and tangent, returning [-1.0, 1.0]
cos(x)	
tan(x)	
asin(x)	Trigonometric arcsine, arccosine and arctangent functions, input [-1.0, 1.0], output [-π/2, π/2]
acos(x)	
atan(x)	
atan2(y, x)	This one's my favorite, output [-π, π]

Math Functions and Type

- C doesn't support function overloading
  - It gives different versions of these functions for float, double and long double
  - Each with a slightly different name.

double	float	long double
sin(x)	sinf(x)	sinl(x)
cos(x)	cosf(x)	cosl(x)
tan(x)	tanf(x)	tanl(x)
asin(x)	asinf(x)	asinl(x)
acos(x)	acosf(x)	acosl(x)
atan(x)	atanf(x)	atanl(x)
atan2(y, x)	atan2f(y, x)	atan2l(y, x)

Exponents, Logs and Roots (oh my)

- But wait, there's more

Function	Description
exp(x)	e^x
exp2(x)	2^x
exp10(x)	10^x
log(x)	$\log_e x$
log2(x)	$\log_2 x$
log10(x)	$\log_{10} x$
pow(x, y)	x^y
sqrt(x)	sqrt(x) … OK, $x^{1/2}$

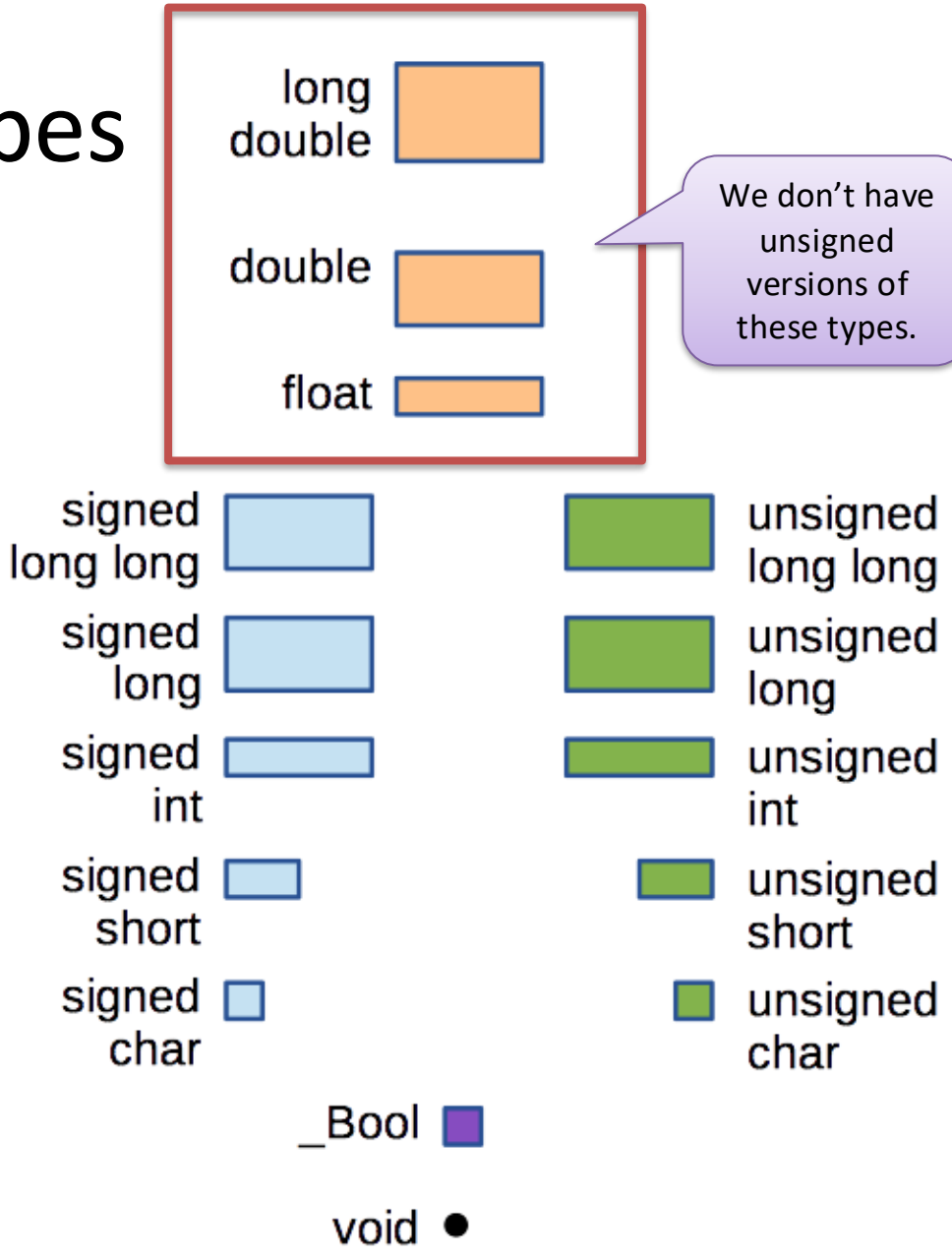
Rounding and Friends

- Even more math functions:

Function	Description
round(x)	nearest integer (as a double)
fabs(x)	absolute value
floor(x)	largest integer no larger than x (as a double)
ceil(x)	smallest integer no less than x (as a double)
fmod(x, y)	floating point mod operation (the % operator only works for integer types)

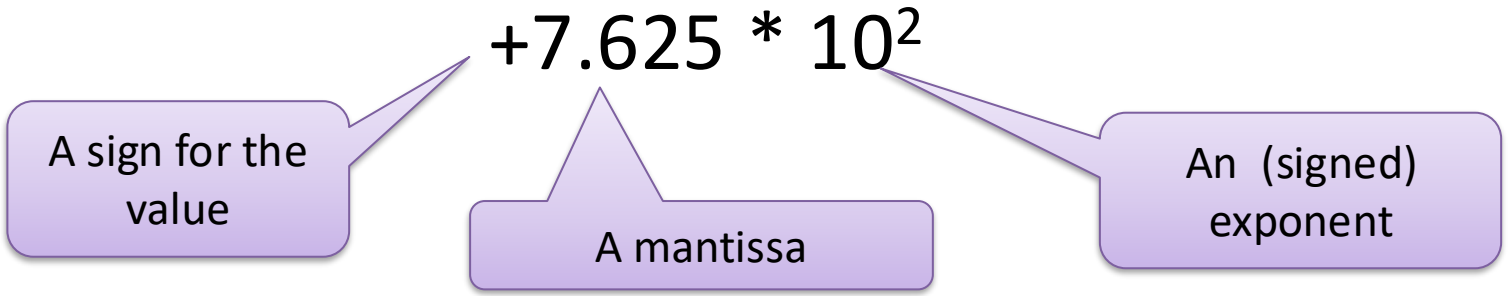
Floating-Point Types

- C has three floating-point types for storing real numbers.
- These have much greater range than integer types
  - But, they typically have to store an approximation of a particular real number.
  - They can't represent every value exactly.



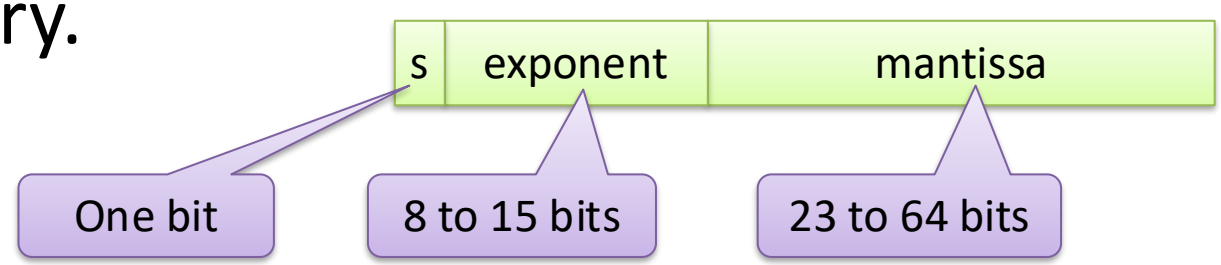
Floating Point Representation

- The C standard doesn't specify how floating-point numbers should be represented
- The IEEE has developed a standard that is very common (IEEE 754)
  - That's what we'll cover.
- Internally, floating point numbers are represented in scientific notation.



Floating Point Representation

- Each part of the representation gets a few bits in memory.



- Size of the exponent determines range of values
- Size of the mantissa determines the precision
- Sign is always 1 bit: 0 → non-negative
  1 → negative

Floating Point Representation

- Internally, floating point numbers are stored in binary.

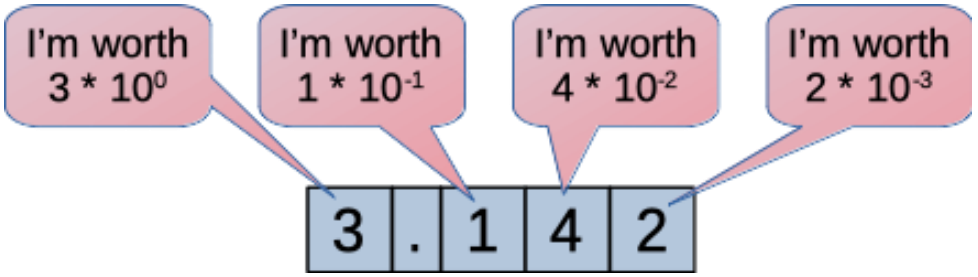
So, the mantissa is really in binary.

$$+111.101 * 10^2$$

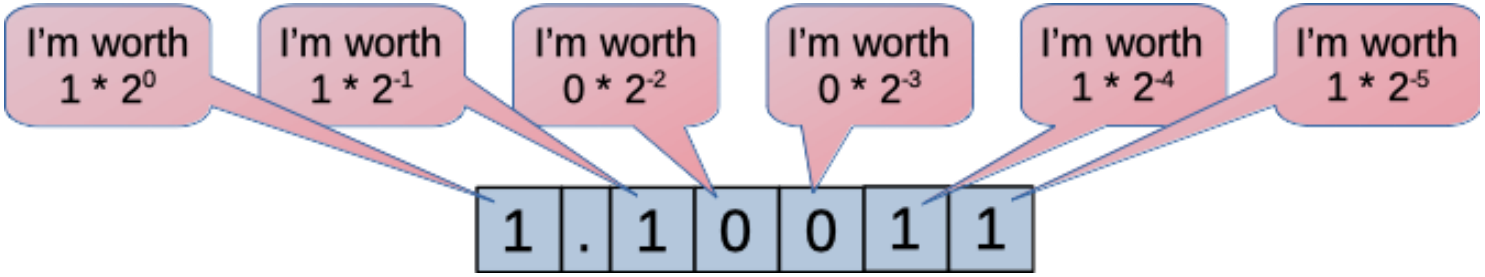
- How do fractions work in binary?

Fractions in Base Two

- For base 10, powers of 10 get smaller to the right of the decimal.

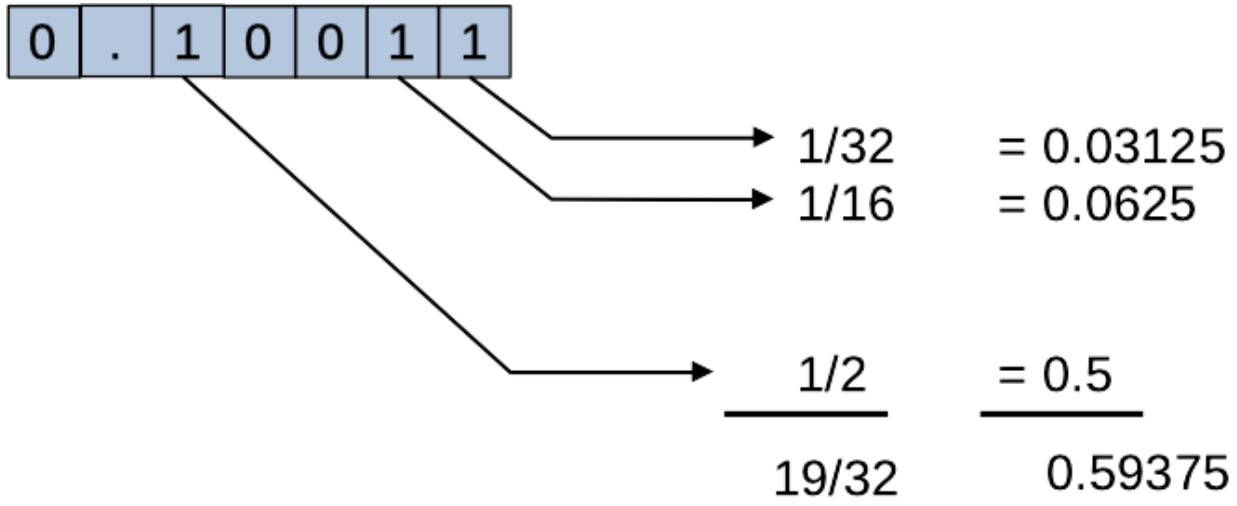


- It's the same in binary, but using powers of 2.



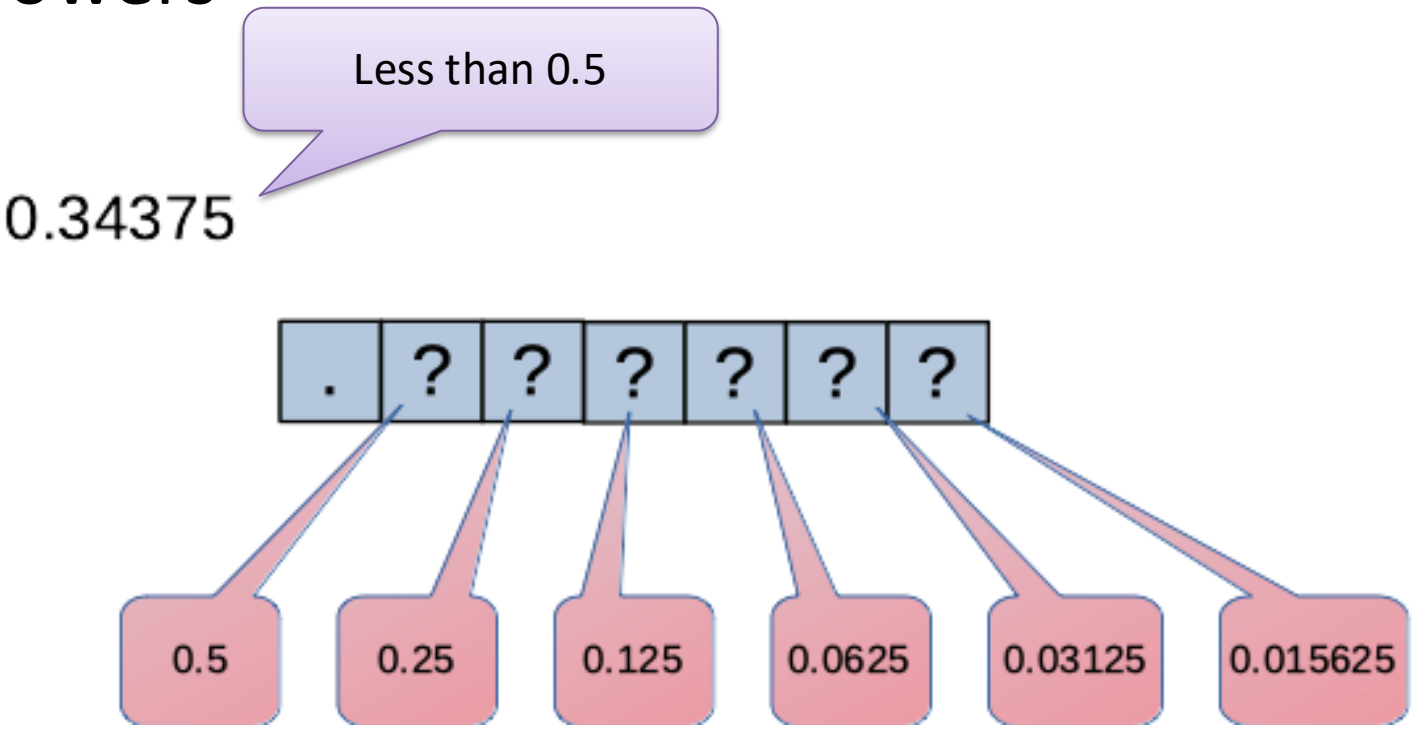
Binary Fractions to Decimal

- To convert a binary fraction to decimal, we can just add up the value of all the 1 bits.



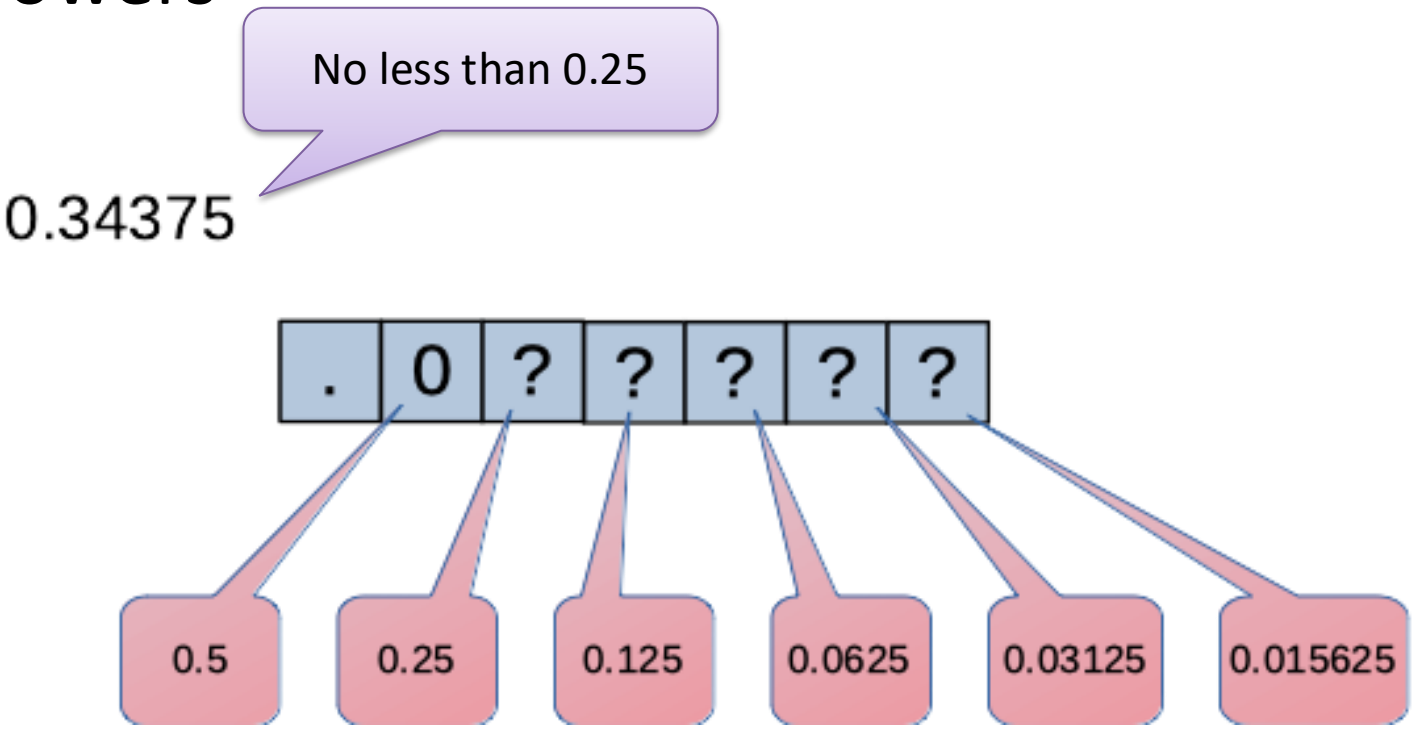
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



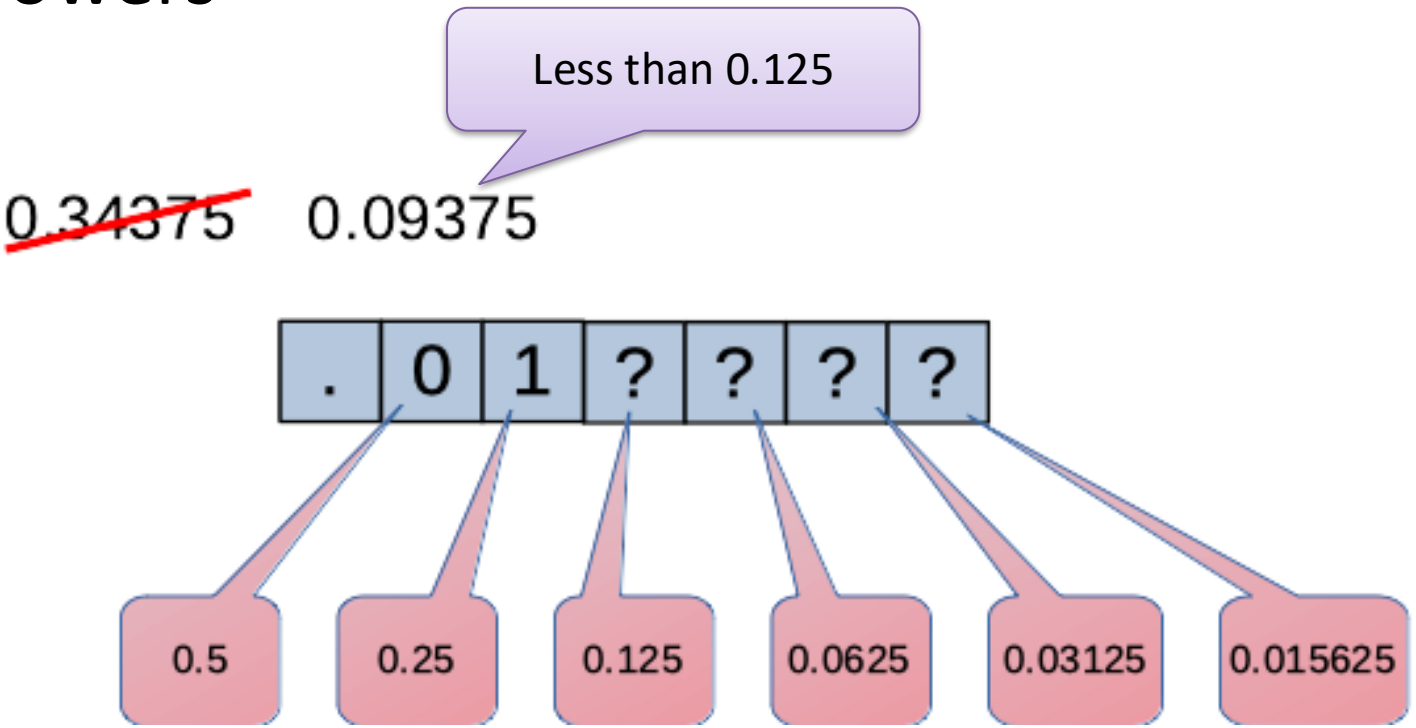
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



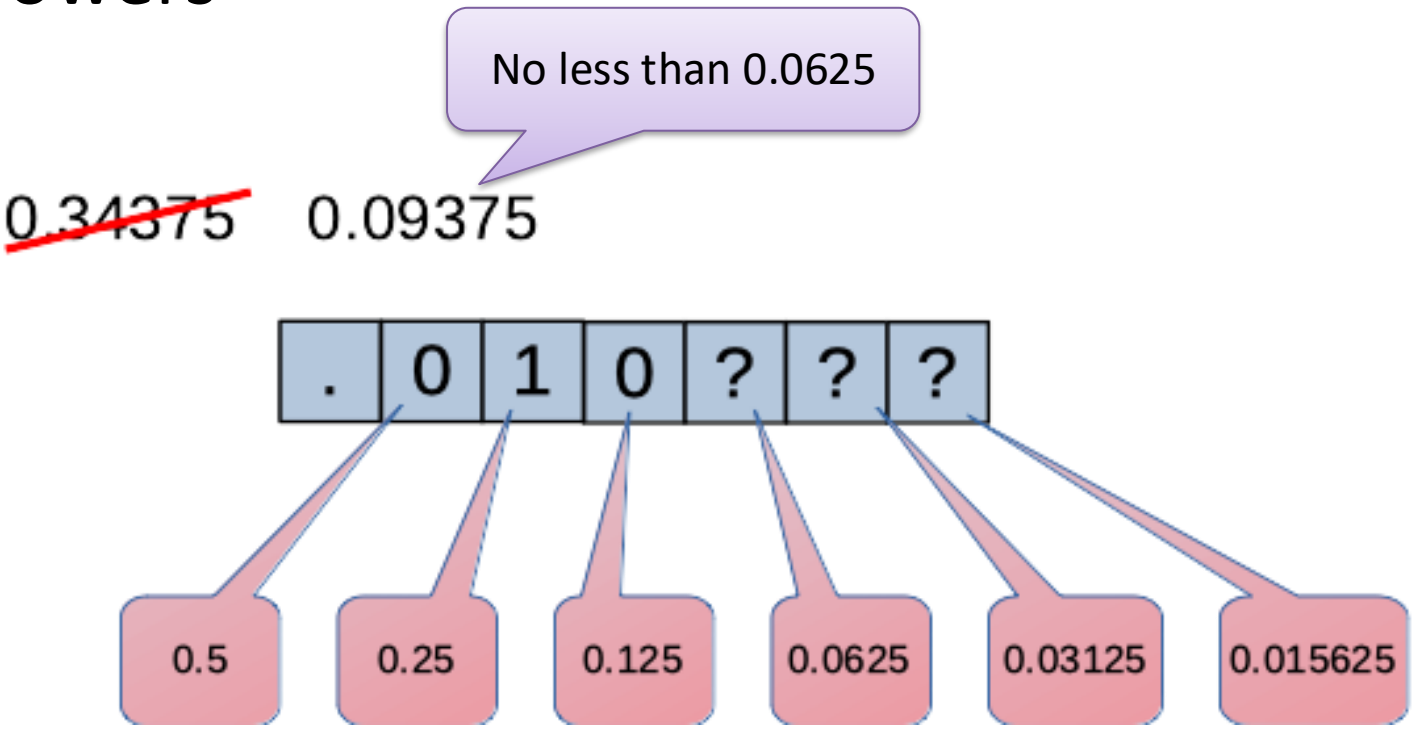
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



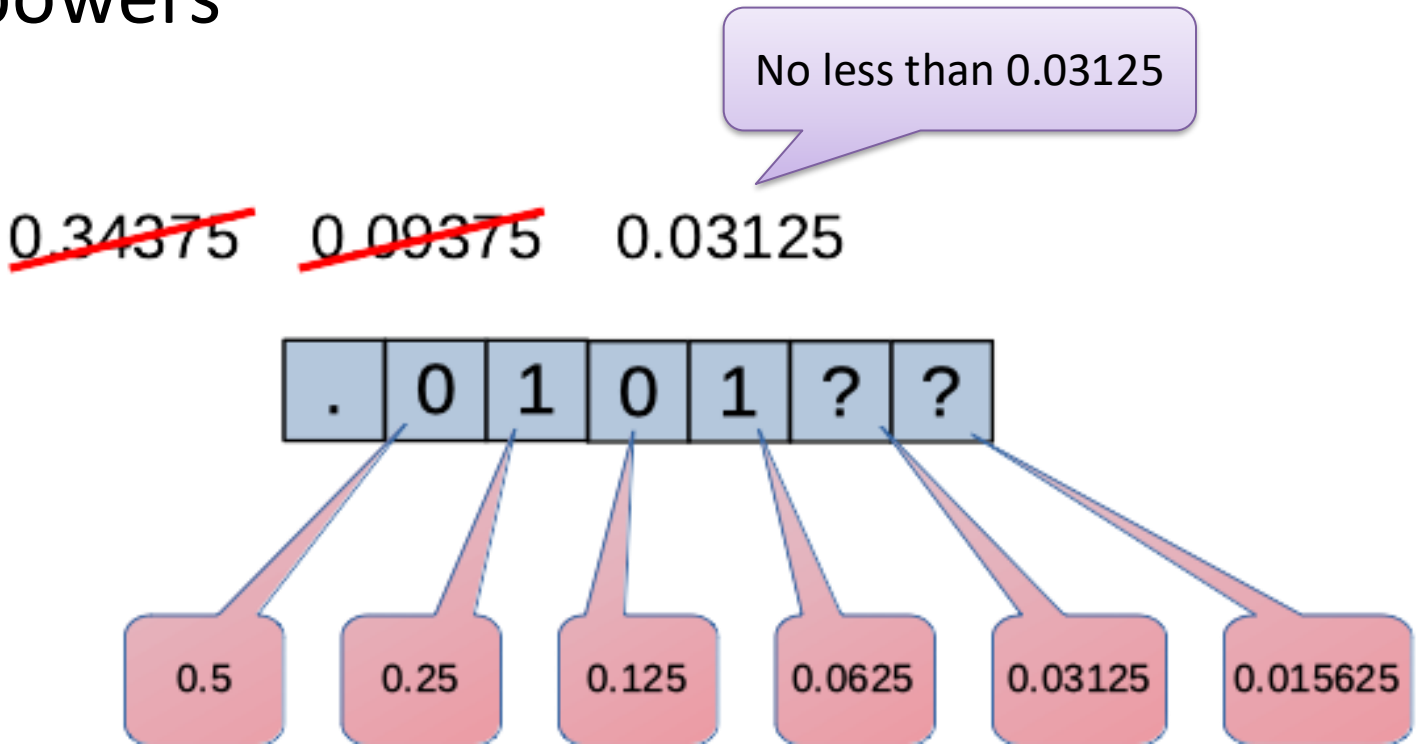
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



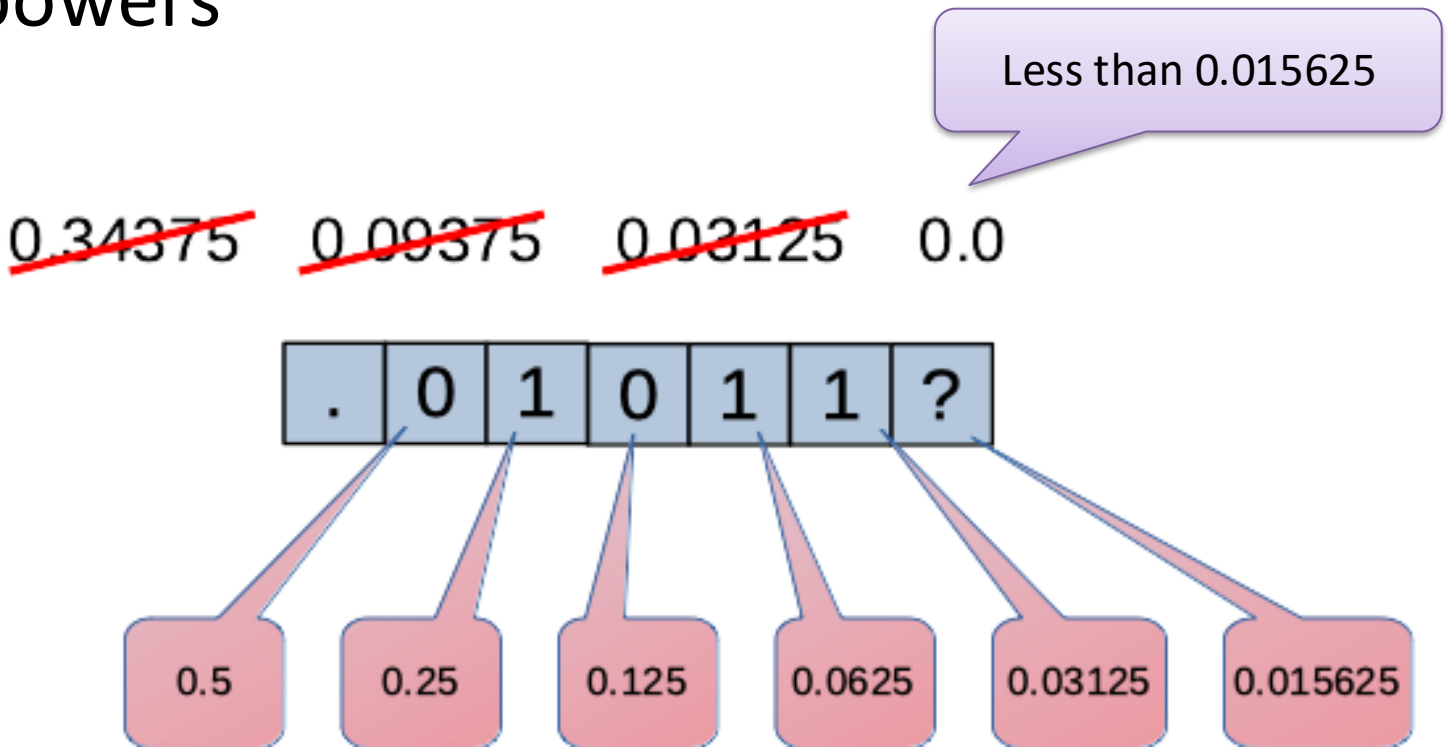
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



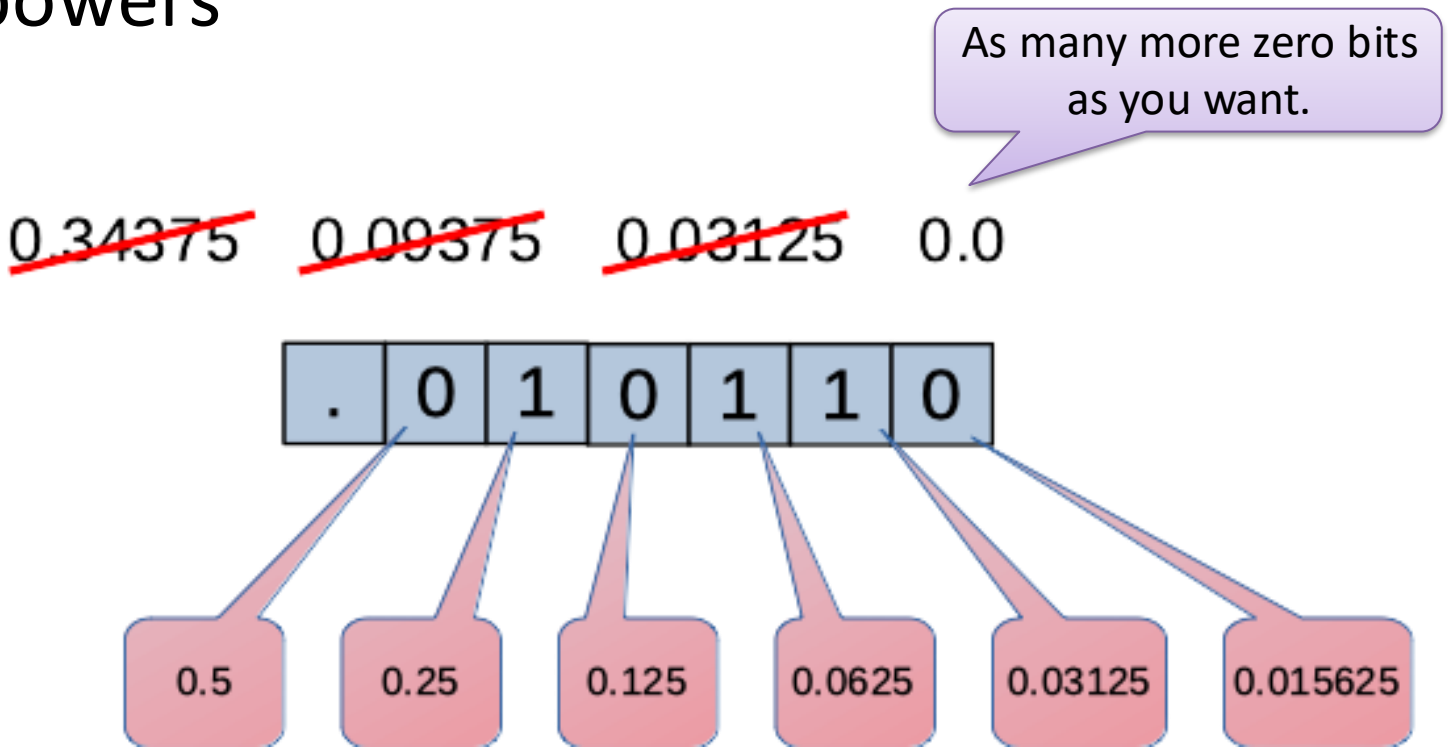
Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



Decimal Fractions to Binary

- To convert to binary, we can use reduction of powers



Floating Point Rounding

- We can see why fractions like 0.1 can't be represented in binary.
- We get a repeating fraction in binary. 0.0001100110011001100
- This makes it easy to write some bad code.

```
for ( double x = 0; x != 1; x += 0.1 ) {
   …;
}
```

This will be an infinite loop.

```
for ( double x = 0; x < 1; x += 0.1 ) {
   …;
}
```

Even this will loop 11 iterations.

Floating Point Representation

- Internally, the exponent is also a power of 2.

$+111.101 * 10^2$  $+10111.110101000 * 2^5$

- With 2 as the base, changing the exponent moves the point left or right.

$+101111.10101000 * 2^4$

$+10111.110101000 * 2^5$

$+1011.1110101000 * 2^6$

Floating Point Representation

- Choosing different exponents, there are lots of ways to write the same number.

$$+10111.110101000 * 2^{5}$$

$$+10.111110101000 * 2^{8}$$

$$+1011111010100.0 * 2^{-3}$$

- The IEEE standard normalizes the mantissa, with the point just after the leftmost 1.

$$+1.0111110101000 * 2^{9}$$

One Bit for Free!

- If the first bit of the mantissa is always a 1 … we don't really have to store it.

$$+1.\mathbf{01111101010 00} * 2^{9}$$

Don't store this bit. Just assume it's a 1.

Store all these bits.

- This is like getting one bit of precision for free.
- But, we will a different way to store 0.0
  (it doesn't contain any 1 bits)

Storing the Exponent

- Of course, the exponent is really stored in binary.

$+1.\mathbf{0111110101000} * 2^{9}$  $+1.\mathbf{0111110101000} * 2^{1001}$

- The exponent is a signed value
  - Could be negative, positive or zero.
  - Should we store this as two's complement number?
  - No. Let's use a different technique.

Biased Exponent

- We'll store the exponent using an unsigned value.
- But, we'll store a value that's larger than the actual exponent.
- For example, excess 127 for an 8-bit exponent.
  - Store a value that's 127 larger than the actual exponent.
  - So, if the actual exponent is zero, we store 01111111_2
  - If the if the actual exponent is 85, we store 11010100_2
  - If the actual exponent is -38, we store 01011001_2

Biased Exponent

- The bias depends on the size of the exponent field.
  - For a 8-bit exponent: excess 127
  - For a 11-bit exponent: excess 1023
  - For a 15-bit exponent: excess 16,383

Floating Point Formats

- Float (IEEE single precision)
  - 1 bit for the sign
  - 23 bits for the mantissa
  - 8 bits for the exponent

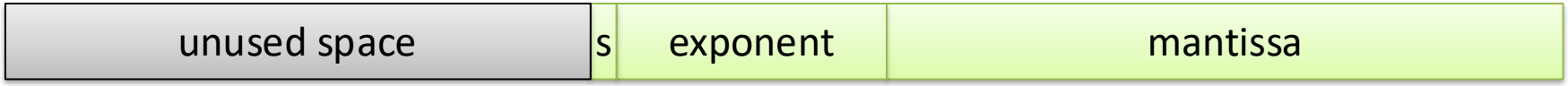


- IEEE double precision:
  - 1 bit for the sign
  - 52 bits for the mantissa
  - 11 bits for the exponent



Floating Point Range

- long double
  - 80-bit format (stored in 16 bytes)
  - 64 bits for the mantissa (including most-significant bit)
  - 15 for the exponent



- Approximate range and precision:

Type	Magnitude range	Precision
float	About 10^{-38} .. 10^{38}	About 7 decimal digits
double	About 10^{-308} .. 10^{308}	About 15 decimal digits
long double	About 10^{-4930}.. 10^{4930}	About 19 decimal digits

Conversion Example

- Example: 3.25 → float
- Convert to binary



11.01

3 in binary

0.25 in binary

- Normalize the mantissa

Had to move the point one bit left

$1.101 * 2^1$

3.25 → float

Sign is non-negative.

$1.101 * 2^1$

We won't store this bit.

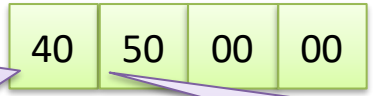
We'll store the exponent as 1 + 127 10000000_2

- In memory, this will look like:



Lots of precision bits we're not really using.

- Or, as bytes in hexadecimal:



Sign bit and exponent almost fit in the first byte.

Last (least-significant) exponent bit here.

Another Conversion Example

- Example: -0.4375 → float
- Convert to binary



-0.0111

- Normalize the mantissa

1 for the sign bit.

$-1.11 * 2^{-2}$

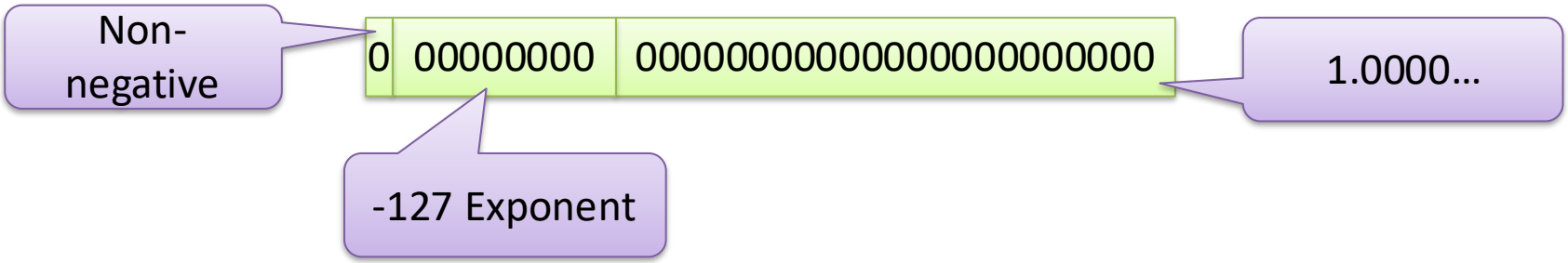
I'll be encoded as: 01111101

- Encode in the 3 fields:



How About Zero?

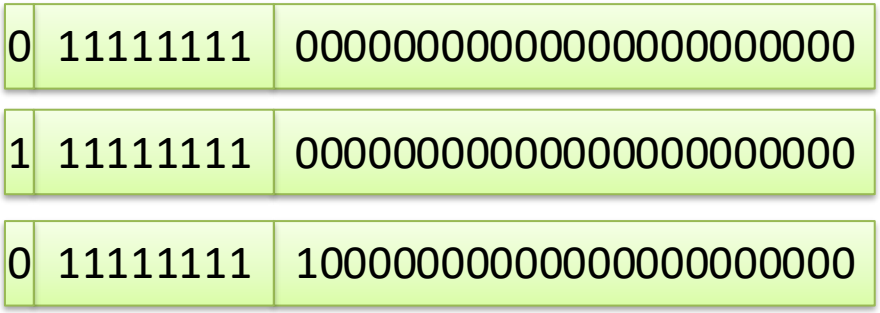
- Normalized mantissa requires a 1 bit.
- How do we represent zero?
- We'll use the smallest possible value we can encode.



- Notice: this is all zeros in memory … like zero for other types.

Special Values

- IEEE standard uses the largest and smallest exponents to represent special values.
- Exponent -127 for zero (and other small values)
- Exponent 128 for:
  - Positive infinity
  - Negative infinity
  - Not-a-Number



Functions for Special Values

- There are math functions to help with special values.

Function	Description
isfinite(x)	Returns true if x is finite.
isinf(x)	Returns true if x is infinite.
isnan(x)	Returns true if x is not-a-number.

One More Conversion Example

- Example: 

→ decimal
- Find the fields: 

So, the value is positive.

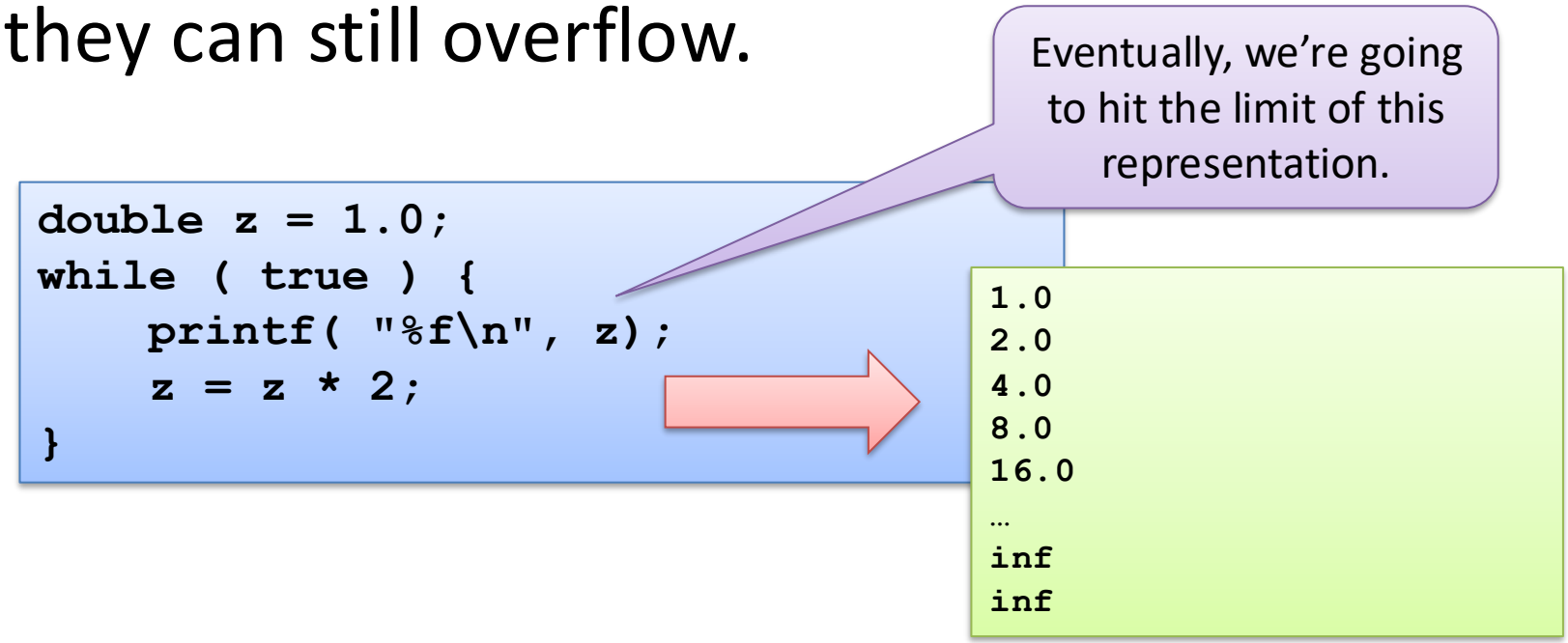
That's 129, so the exponent is really 129 – 127 = 2.

That's a mantissa of: 1.0111

- That's. $1.0111 * 2^2 = 101.11 * 2^0$
- In decimal, that's 5.75

Floating-Point Overflow

- Floating point values have a wide range … but they can still overflow.



- And, there's another floating point behavior we didn't have to worry about with integers.

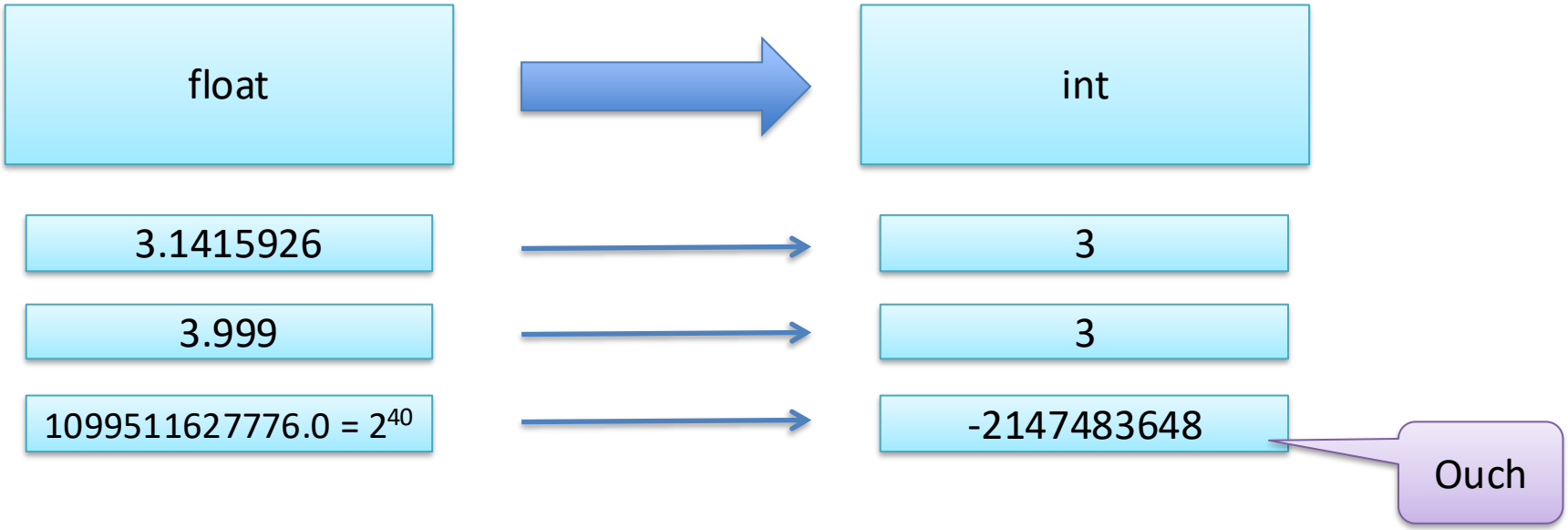
Underflow

- Floating point numbers can represent magnitudes only so small
- double and long double have extra capacity, but, still, there's a limit
- Falling below this low-magnitude limit is called *underflow*

```
double z = 1.0;
while ( z != 0.0 ) {
    printf( "%f\n", z);
    z = z * 0.5;
}
```

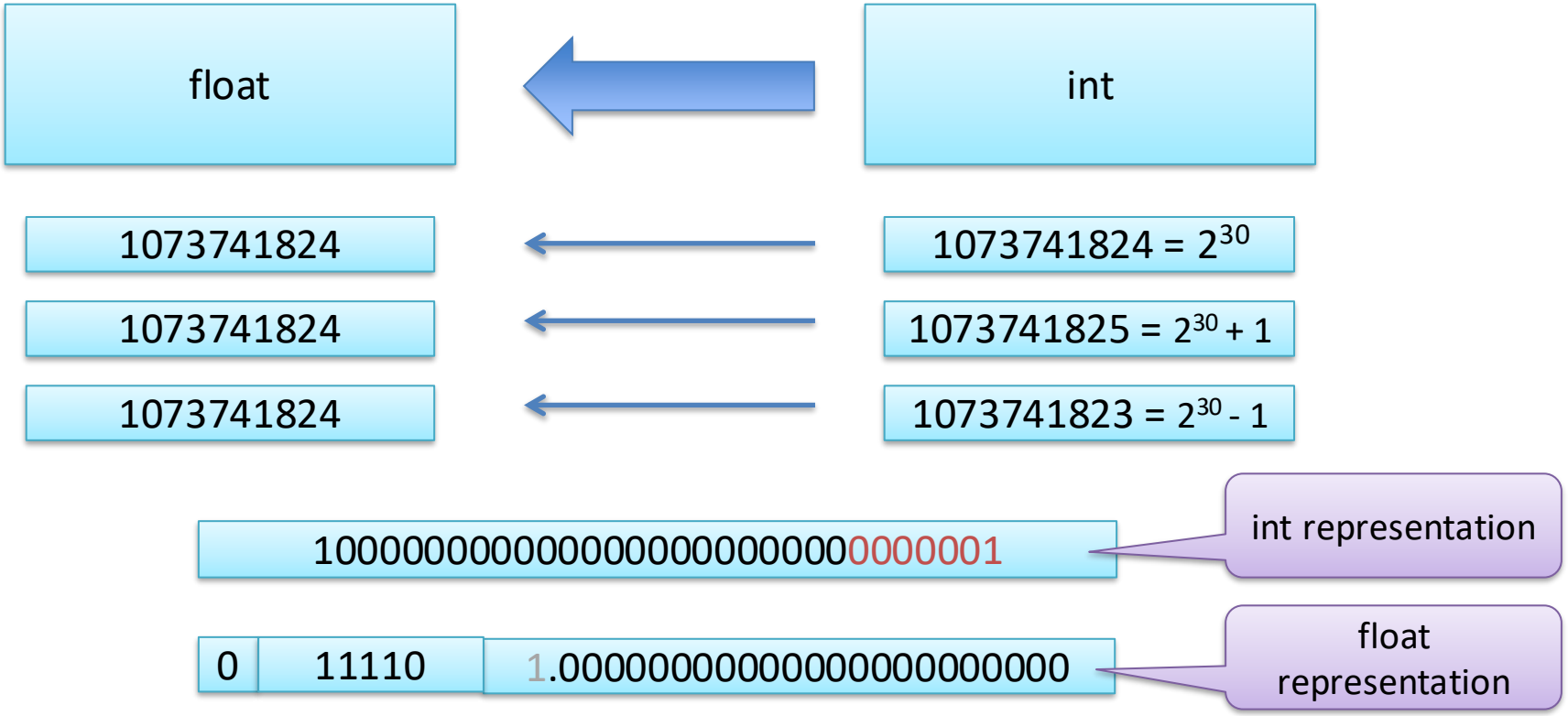
Floating-Point / Integer Conversion

- C has simple rules for converting from real to integer types.
  - Truncate fractional part.
  - The integer gets the remaining, whole number part.
  - There could definitely be some overflow here.



Floating-Point / Integer Conversion

- Integer to Floating Point
  - Approximate with the closest floating point value



Floating-Point / Integer Conversion

- In general, floating point types can represent any small integer values exactly …
- … but not all really big values.
- On the common platform there are potential rounding errors:

